
AN INTEGRATED UVM-TLM CO-SIMULATION FRAMEWORK FOR RISC-V FUNCTIONAL VERIFICATION AND PERFORMANCE EVALUATION

A PREPRINT

Ruizhi Qiu

School of Integrated Circuits Science and Engineering
Beihang University, Beijing, China
qiuruizhi02@buaa.edu.cn

Yang Liu

School of Integrated Circuits Science and Engineering
Beihang University, Beijing, China
liuyang_me@buaa.edu.cn

ABSTRACT

The burgeoning RISC-V ecosystem necessitates efficient verification methodologies for complex processors. Traditional approaches often struggle to concurrently evaluate functional correctness and performance, or balance simulation speed with modeling accuracy. This paper introduces an integrated co-simulation framework leveraging Universal Verification Methodology (UVM) and Transaction-Level Modeling (TLM) for RISC-V processor validation. We present a configurable UVM-TLM model (*vmodel*) of a superscalar, out-of-order RISC-V core, featuring key microarchitectural modeling techniques such as credit-based pipeline flow control. This environment facilitates unified functional verification via co-simulation against the Spike ISA simulator and enables early-stage performance assessment using benchmarks like CoreMark, orchestrated within UVM. The methodology prioritizes integration, simulation efficiency, and acceptable fidelity for architectural exploration over cycle-level precision. Experimental results validate functional correctness and significant simulation speedup over RTL approaches, accelerating design iterations and enhancing verification coverage.

Keywords RISC-V · Processor Modeling · UVM · TLM · Co-verification · Functional Verification · Performance Evaluation · Flow Control

1 Introduction

The rapid proliferation of the RISC-V Instruction Set Architecture (ISA) [1] offers design flexibility but also presents significant verification challenges, especially for complex, customizable processors incorporating features like superscalar and out-of-order execution [2]. Traditional Register-Transfer Level (RTL) simulations, while cycle-accurate, are prohibitively slow for agile development [3, 4]. High-level architectural simulators such as gem5 [5] or QEMU [6] offer speed but may lack microarchitectural detail or seamless integration with industry-standard verification flows like the Universal Verification Methodology (UVM) [7, 8]. This often leads to a decoupling of functional verification and performance analysis [9].

To address these limitations, this paper proposes an integrated co-simulation framework based on the synergy of UVM [10, 11] and Transaction-Level Modeling (TLM) [12, 13]. UVM provides a structured, reusable infrastructure for comprehensive functional verification [14], while TLM accelerates simulation through abstracted communication [15]. Our framework utilizes these to create a platform where a detailed RISC-V processor model undergoes simultaneous functional validation against a golden reference (Spike ISA simulator) and performance characterization, all within a cohesive UVM environment. The core of our contribution is this **integrated verification and evaluation methodology and its enabling framework**, which emphasizes a balance between simulation throughput and acceptable modeling fidelity for early-stage architectural exploration.

The main contributions are:

- The design and implementation of an integrated UVM-TLM co-simulation framework tailored for RISC-V processor verification, facilitating concurrent functional validation and performance trend analysis.
- A detailed UVM-TLM model (*vmodel*) of a superscalar, out-of-order RISC-V core, demonstrating key modeling techniques for complex microarchitectural features, including a credit-based backpressure flow control for pipeline stages.
- Experimental validation of the framework’s effectiveness in achieving high functional coverage, identifying microarchitectural issues early, and providing significant simulation speedup compared to RTL approaches, while enabling relative performance comparisons for design space exploration.

The remainder of this paper is organized as follows: Section 2 briefly covers background technologies and related work. Section 3 details the proposed framework and modeling aspects. Section 4 outlines the verification and evaluation procedures. Section 5 presents experimental results. Finally, Section 6 concludes and suggests future work.

2 Background and Related Work

Our approach leverages UVM and TLM. UVM is the industry standard for SystemVerilog-based functional verification, promoting modularity and reusability [10]. TLM, particularly TLM-2.0 [16], abstracts inter-module communication to transactions, significantly speeding up system-level simulations [13]. The synergy of UVM and TLM has been recognized for creating high-performance verification environments, especially for complex SoCs [17, 18].

Existing RISC-V verification efforts include formal methods [19], dedicated test generators, and UVM-based functional testbenches [20]. Performance modeling often relies on tools like gem5 [21] or custom SystemC/TLM models, typically separate from UVM flows. While UVM/TLM integration has been applied to SoC interconnects [22] or cache coherence [23], a comprehensive UVM-TLM framework for a complex, out-of-order RISC-V core model that deeply integrates functional verification with early performance analysis and detailed microarchitectural modeling techniques (like realistic pipeline flow control) is less explored. Our work aims to address this by providing a unified and efficient solution.

3 Proposed UVM-TLM Co-Simulation Framework

The core of our work is a UVM-TLM based framework for concurrent functional verification and performance assessment of RISC-V processors. It features a detailed, configurable *vmodel* of a superscalar, out-of-order RISC-V core.

3.1 Framework Architecture

The framework (Figure 1) uses a layered UVM architecture integrating the *vmodel*, a golden reference model (*cmodel* - Spike [1] via DPI-C), and UVM verification components (testbench, environment, agents, sequencer, comparer/scoreboard).

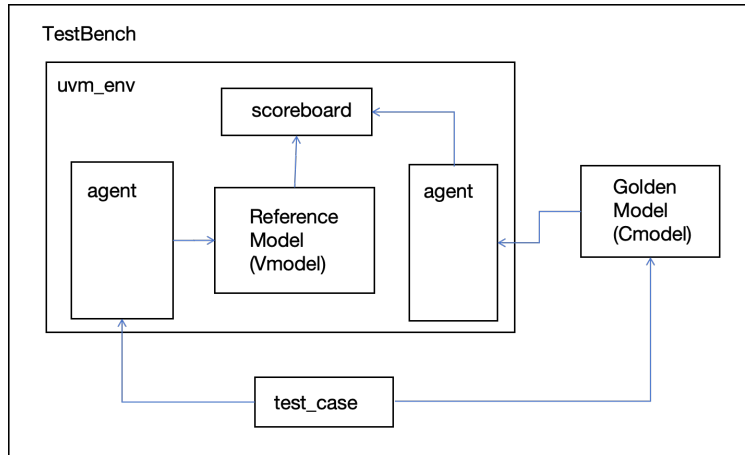


Figure 1: Architecture of the UVM-TLM Co-Simulation Framework.

3.2 Key Modeling Aspects of *vmodel*

The *vmodel* is not a black box but reflects key microarchitectural concepts:

- **Modular TLM-Based Communication:** Internal blocks (e.g., Frontend, Backend, Load/Store Unit (LSU), L1 Data cache) communicate via standard UVM-TLM 2.0 interfaces. Instructions, memory requests, and control signals are encapsulated as TLM transactions.
- **Abstracted Pipeline Modeling with Credit-Based Flow Control:** The *vmodel* simulates key pipeline stages (fetch, decode, rename, dispatch, issue, execute, commit). Crucially, to manage transaction flow between pipelined stages (e.g., Frontend to Backend) in the asynchronous TLM environment, a credit-based backpressure protocol is implemented. Downstream modules grant "credits" to upstream modules via dedicated TLM channels, indicating available buffer space. An upstream module consumes a credit upon sending a transaction and stalls if no credits are available. This models realistic pipeline backpressure due to resource contention, enhancing behavioral accuracy beyond simple sequential execution.

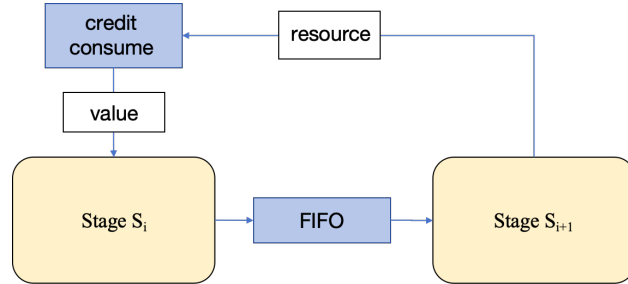


Figure 2: Conceptual diagram of credit-based flow control between pipeline stages S_i and S_{i+1} .

- **Integrated Functional and Performance Representation:** TLM transactions can carry approximate timing. The model tracks resource utilization and, importantly, counts retired instructions at the commit stage for accurate performance metrics.
- **Hierarchical and Configurable Structure:** The SystemVerilog codebase is logically organized. The *vmodel* is configurable via UVM's configuration database, allowing parameters like issue width or buffer sizes to be varied for architectural exploration.

The frontend (instruction fetch, decode) and backend (rename, dispatch, issue, execution units, commit) are modeled as distinct UVM components interacting via TLM. The LSU handles memory operations, interfacing with an L1D cache model, which in turn connects to a main memory model, all using TLM.

3.3 Co-Simulation Verification Mechanism

Functional correctness is verified by co-simulating *vmodel* and *cmodel* with identical instruction streams (Figure 3). The UVM Comparer module performs a lock-step comparison of architectural states (PC, GPRs, CSRs) upon instruction commitment in *vmodel*, flagging any discrepancies.

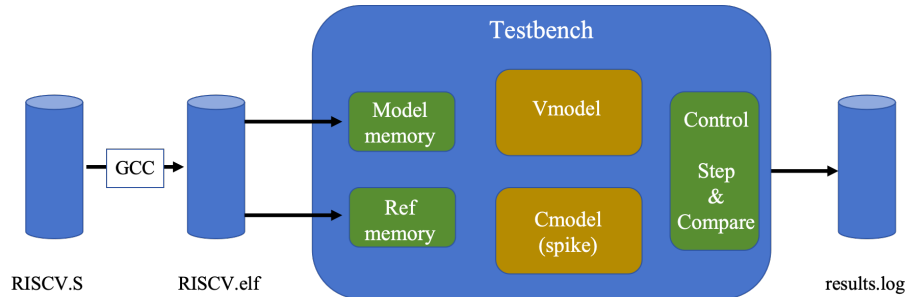


Figure 3: Functional verification flow using co-simulation.

4 Integrated Verification and Evaluation Methodology

The framework enables a cohesive workflow for both functional verification and performance evaluation.

4.1 Functional Verification Procedure

Standardized test suites (e.g., *riscv-tests*) are compiled for the *vmodel*'s target ISA. Binaries are loaded into a shared memory model (potentially via backdoor access for speed). The UVM testbench drives execution on both *vmodel* and *cmodel*, with the Comparer automatically performing state comparison upon instruction retirement.

4.2 Performance Evaluation Procedure

Using the same UVM environment, benchmarks like CoreMark [24] are run on *vmodel*. Key Performance Indicators (KPIs) like Instructions Per Cycle (IPC) are derived from:

- **Execution Cycles:** Precisely tracked by the UVM environment for the *vmodel* core.
- **Retired Instructions:** Accurately counted within *vmodel* as instructions are architecturally committed in the final pipeline stage, providing a reliable measure of useful work. This count is propagated to the UVM environment.

A dedicated UVM test sequence manages the performance run, including configuring microarchitectural parameters of *vmodel*, loading the benchmark, initiating/halting counters, and calculating KPIs (e.g., $IPC = \text{Retired Instructions} / \text{Execution Cycles}$).

4.3 Methodological Emphasis and Fidelity

The primary objective is **integrated assessment and relative performance analysis**. The UVM-TLM model, by design (e.g., approximate timing, simplified L1 cache model), does not yield cycle-accurate results comparable to RTL. Its value lies in significant simulation speedup, enabling rapid evaluation of *performance trends and the relative impact* of different microarchitectural configurations (e.g., varying issue width, execution unit count) early in the design phase.

5 Results and Discussion

Experiments were conducted using Synopsys VCS on a Linux server environment with Intel Xeon Gold CPUs.

5.1 Functional Verification Efficacy

The *vmodel* (configured for RV64IMAFDC) successfully passed all relevant *riscv-tests* (Table 1), confirming ISA-level functional correctness.

Table 1: RISC-V Compliance Test Suite (*riscv-tests*) Execution Summary

Test Subset	Target ISA	Executed Cases	Pass Rate
RV64I	RV64IMAFDC	81	100%
M, A, F, D, C Ext.	RV64IMAFDC	97	100%

More significantly, the UVM environment's white-box observability and functional coverage collection (achieving >97% for key microarchitectural mechanisms) proved crucial for in-depth validation. This approach enabled the detection and diagnosis of subtle microarchitectural bugs that are often masked in purely ISA-level "black-box" comparisons. For instance, through detailed UVM monitoring and coverage analysis of internal pipeline states and data paths, we identified issues such as: (1) an intermittent data error due to data forwarding path contention in specific dual-issue scenarios under high load, and (2) an operand partitioning error during the decoding of certain non-standard instruction variants. Discovering such bugs at the TLM abstraction level, facilitated by UVM's introspection capabilities, offers a significant time saving (estimated 4-6 weeks in our project timeline) compared to their potential discovery much later in the RTL verification phase, thereby greatly improving overall design and verification efficiency.

5.2 Performance Assessment and Simulation Efficiency

Performance evaluation was conducted using CoreMark 1.0 (compiled with GCC 9.4.0, -O2). The *vmodel* baseline configuration was a single core, 4-wide issue, 128-entry ROB, 32KB 8-way L1D cache (abstracted), and a BTB+TAGE-like branch predictor. Results are compared to a Xiangshan Nanhu processor reference (Table 2).

Table 2: CoreMark 1.0 Performance Comparison

Model/System	Retired Instr.	Exec. Cycles	IPC	CoreMark/MHz
<i>vmodel</i> (This Work)	≈24.5 B	≈10.2 B	≈2.40	≈8.90
Xiangshan Nanhu (Ref.)	≈31.2 B	≈13.0 B	≈2.40	≈7.81

The *vmodel*’s CoreMark/MHz score exhibits an approximate 14.1% deviation from the Xiangshan reference. This level of accuracy is within the acceptable range for early-stage TLM models where the primary goal is architectural exploration and relative performance trend analysis, rather than absolute cycle-level precision [16, 25, 26]. The deviation primarily stems from abstractions in the TLM model, notably the simplified L1 cache implementation (currently pass-through without detailed hit/miss latency modeling), generalized pipeline stage timing, and idealized resource contention models. Despite these abstractions, the model effectively captures relative performance variations when microarchitectural parameters (e.g., execution unit counts, queue depths) are modified, validating its utility for design space exploration. Some specialized high-accuracy TLM simulations have reported lower error margins [27], indicating avenues for future refinement.

The most significant advantage of the UVM-TLM framework is its **simulation efficiency**. Executing the full CoreMark benchmark on the *vmodel* took approximately 10-15 minutes on our simulation platform. In stark contrast, an equivalent simulation run on a corresponding RTL model is estimated to require 10-12 hours. This orders-of-magnitude speedup dramatically accelerates design-verify-debug cycles, enabling more extensive testing and rapid exploration of architectural alternatives early in the design process.

6 Conclusion and Future Work

This paper presented an integrated UVM-TLM co-simulation framework designed for the efficient verification and early performance evaluation of complex RISC-V processors. Our approach, featuring a detailed UVM-TLM model (*vmodel*) of a superscalar, out-of-order RISC-V core, facilitates concurrent functional validation against the Spike ISA simulator and performance trend analysis using standard benchmarks. Key modeling aspects, such as credit-based pipeline flow control, were discussed.

Experimental results demonstrated the framework’s ability to ensure ISA-level functional correctness and, more importantly, leverage UVM’s capabilities for in-depth microarchitectural validation, leading to early detection of subtle design issues. While the TLM abstraction level results in a deviation in absolute performance metrics (approx. 14.1% for CoreMark/MHz against a reference), the model provides valuable relative performance insights and achieves orders-of-magnitude simulation speedup compared to traditional RTL methods. This significantly accelerates architectural exploration and design iteration.

Future work will focus on: (1) **Enhancing Model Fidelity** by implementing a more detailed L1/L2 cache hierarchy with accurate hit/miss logic and realistic timing, and refining timing models for other critical pipeline components. (2) **Exploring Model-Driven RTL Development** by investigating how the structured TLM model can guide or partially automate the generation of RTL code. (3) **Expanding Verification Scope** to include advanced microarchitectural features, multi-core systems, and security aspects.

The proposed UVM-TLM framework offers a practical and effective methodology for navigating the complexities of modern RISC-V processor design and verification.

References

- [1] David A Patterson and John L Hennessy. *Computer organization and design RISC-V edition: the hardware software interface*. Morgan kaufmann, 2017.
- [2] Sallar Ahmadi-Pour, Vladimir Herdt, and Rolf Drechsler. Constrained random verification for risc-v: overview, evaluation and discussion. In *MBMV 2021; 24th Workshop*, pages 1–8. VDE, 2021. rv-verification.

- [3] Nursultan Kabytkas, Tommy Thorn, Shreesha Srinath, Polychronis Xekalakis, and Jose Renau. Effective processor verification with logic fuzzer enhanced co-simulation. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 667–678, 2021.
- [4] Donggyu Kim, Adam Izraelevitz, Christopher Celio, Hokeun Kim, Brian Zimmer, Yunsup Lee, Jonathan Bachrach, and Krste Asanović. Strober: Fast and accurate sample-based energy simulation for arbitrary rtl. *ACM SIGARCH Computer Architecture News*, 44(3):128–139, 2016.
- [5] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. The gem5 simulator. *ACM SIGARCH computer architecture news*, 39(2):1–7, 2011.
- [6] Slawomir Chylek. Collecting program execution statistics with qemu processor emulator. In *2009 International Multiconference on Computer Science and Information Technology*, pages 555–558. IEEE, 2009.
- [7] Nils Przigoda, Jannis Stoppe, Julia Seiter, Robert Wille, and Rolf Drechsler. Verification-driven design across abstraction levels: A case study. In *2015 Euromicro Conference on Digital System Design*, pages 375–382. IEEE, 2015.
- [8] Edel Díaz, Raúl Mateos, Emilio J Bueno, and Rubén Nieto. Enabling parallelized-qemu for hardware/software co-simulation virtual platforms. *Electronics*, 10(6):759, 2021.
- [9] Gaurav Bhatnagar and Courtney Fricano. Product life cycle of interconnect bus: A portable stimulus methodology for performance modeling design verification and post-silicon validation. In *Design and Verification Conference US 2019 (DVCon US 2019)*, 2019.
- [10] Hannibal Height. *A practical guide to adopting the universal verification methodology (UVM)*. Lulu. com, 2012.
- [11] NB Harshitha, YG Praveen Kumar, and MZ Kurian. An introduction to universal verification methodology for the digital design of integrated circuits (ic’s): A review. In *2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS)*, pages 1710–1713. IEEE, 2021.
- [12] Alexander W Rath, Volkan Esen, and Wolfgang Ecker. A transaction-oriented uvm-based library for verification of analog behavior. In *2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 806–811. IEEE, 2014.
- [13] Lukai Cai and Daniel Gajski. Transaction level modeling: an overview. In *Proceedings of the 1st IEEE/ACM/IFIP international conference on Hardware/software codesign and system synthesis*, pages 19–24, 2003.
- [14] Wei Ni and Jichun Zhang. Research of reusability based on uvm verification. In *2015 IEEE 11th International Conference on ASIC (ASICON)*, pages 1–4. IEEE, 2015.
- [15] Nourddine Abid, Wissem Chouchene, Brahim Attia, Abdelrim Zitouni, and Rached Tourki. Design and performance evaluation of on chip network with transaction level modeling. In *ICM 2011 Proceeding*, pages 1–6. IEEE, 2011.
- [16] Open SystemC Initiative (OSCI). Requirements specification for TLM 2.0 version 1.1. Requirements Specification Version 1.1, Open SystemC Initiative (OSCI), September 2007. Accessed on 2025-04-28.
- [17] Adam Donlin. Transaction level modeling: Flows and use models. In *Proceedings of the 2nd IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS ’04)*, pages 75–80, September 2004.
- [18] Young-Nam Yun, Jae-Beom Kim, Nam-Do Kim, and Byeong Min. Beyond uvm for practical soc verification. In *2011 International SoC Design Conference*, pages 158–162. IEEE, 2011.
- [19] Lennart Weingarten, Kamalika Datta, Abhoy Kole, and Rolf Drechsler. Complete and efficient verification for a risc-v processor using formal verification. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6. IEEE, 2024.
- [20] J. Redi, T. Skinner, J. Schelle, O. Ergin, V. Govindan, V. Chandra, N. Stephens, R. Hogervorst, B. Flachs, and V. Nagarajan. RISC-V-DV: An open source random instruction stream generator for RISC-V verification. In *Proceedings of the 2019 Design and Verification Conference and Exhibition U.S. (DVCon U.S.)*, pages 1–10, Feb 2019.
- [21] Tuan Ta, Lin Cheng, and Christopher Batten. Simulating multi-core risc-v systems in gem5. In *Workshop on Computer Architecture Research with RISC-V*, 2018.
- [22] Pranay Samanta, Deepak Chauhan, Sujay Deb, and Piyush Kumar Gupta. Uvm based sbus verification ip for verifying soc architectures. In *18th International Symposium on VLSI Design and Test*, pages 1–2. IEEE, 2014.

- [23] Barada P Biswal, Anurag Singh, and Balwinder Singh. Cache coherency controller verification ip using systemverilog assertions (sva) and universal verification methodologies (uvm). In *2017 11th international conference on intelligent systems and control (ISCO)*, pages 21–24. IEEE, 2017.
- [24] Shay Gal-On and Markus Levy. Exploring coremark a benchmark maximizing simplicity and efficacy. *The Embedded Microprocessor Benchmark Consortium*, 6(23):87, 2012.
- [25] Sudeep Pasricha and Nikil Dutt. *On-chip communication architectures: system on chip interconnect*. Morgan Kaufmann, 2010.
- [26] T. Kogel. Using the new TLM-2.0 standard for the creation of virtual platforms for ESL design. Presentation Slides at the Multiprocessor System-on-Chip Forum (MPSoC Forum), July 2008. Accessed on 2025-04-28.
- [27] SoC Design Laboratory, Konkuk University. Description of AccTLMSim pre-RTL simulator on SoC design laboratory talks webpage. Webpage, 2019.